Studio Nx

JS 09 — setFieldState()

2026-10-07

JS 09 — `setFieldState()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS · PRD 01 FR-101 · FR-102
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	08 `logout()` (P1-02 끝) · 10 `initPasswordToggles()`

1. 하는 일

입력칸 하나의 **상태**(초록 · 빨강 · 기본)와 아래 **문구**를 바꾼다.

호출	화면 (join.html 이메일 칸)
`setFieldState(field, 'ok', '✓ Available')`	밑줄 **초록**, 아래 초록 글 "✓ Available"
`setFieldState(field, 'error', 'This email is already in use.')`	밑줄 **빨강**, 아래 빨강 글. 화면 읽기 프로그램은 "잘못된 값" 으로 읽음
`setFieldState(field, null)`	밑줄 기본 색(보라), 문구는 처음 HTML 의 안내 글로 (join.html 은 빈 글자)

`field` 는 입력칸을 감싼 `.field` 요소다 (예: `#join-email-field`).

```
<div class="field" id="join-email-field">          ← is-ok / is-error 가 여기 붙음
  <input class="field__control" id="join-email" …>  ← aria-invalid="true" 가 여기 붙음
  <p class="field__msg" id="join-email-msg"></p>     ← 문구가 여기 들어감
</div>
```

2. 언제 쓰나

P1-07 의 다른 함수들이 칸마다 결과를 보여 줄 때 부른다. 가장 먼저 만든 이유다.

부르는 곳 (P1-07)	칸	예
`initDuplicateChecks()`	이메일 · 닉네임	Check → ✓ Available / This email is already in use.
`initPasswordRules()`	비밀번호 · 비밀번호 확인	Use only English letters, numbers and symbols. / Passwords don't match.
`initJoinForm()`	서버가 422 로 돌려준 칸	`details.nickname` → 닉네임 칸 빨강

색은 `forms.css` 가 정한다 — 이 함수는 **클래스와 속성만** 바꾼다.

붙이는 것	`forms.css` 가 하는 일
`.field.is-ok`	밑줄 · 문구 초록 (`--c-ok`)
`.field.is-error` + `aria-invalid="true"`	밑줄 · 문구 빨강 (`--c-error`)

3. 코드

import (파일 맨 위)

```
// (없음)
```

`auth-ui.js` 는 아직 import 가 없다 (다음 함수들에서 `api.js` · `auth.js` 를 가져올 예정). **이 함수는 import 가 필요 없다** — `querySelector` · `classList` · `dataset` 은 브라우저 기본 기능이다.

함수

```
export function setFieldState(field, state, message) {
  if (!field) return;
  const control = field.querySelector('.field__control');
  const msg = field.querySelector('.field__msg');

  field.classList.remove('is-ok', 'is-error');
  if (control) control.removeAttribute('aria-invalid');

  if (state === 'ok') {
    field.classList.add('is-ok');
  } else if (state === 'error') {
    field.classList.add('is-error');
    if (control) control.setAttribute('aria-invalid', 'true');
  }

  if (msg) {
    if (msg.dataset.hint === undefined) msg.dataset.hint = msg.textContent; // 처음 안내 글을 기억
    msg.textContent = message === undefined ? msg.dataset.hint : message;
  }
}
```

최신 문법으로 쓰면 (공부용 — 실제 코드는 위의 전통 문법)

```
export function setFieldState(field, state, message) {
  if (!field) return;
  const control = field.querySelector('.field__control');
  const msg = field.querySelector('.field__msg');

  field.classList.toggle('is-ok', state === 'ok');
  field.classList.toggle('is-error', state === 'error');
  if (state === 'error') control?.setAttribute('aria-invalid', 'true');
  else control?.removeAttribute('aria-invalid');

  if (msg) {
    msg.dataset.hint ??= msg.textContent;
    msg.textContent = message ?? msg.dataset.hint;
  }
}
```

최신 문법	실제 코드 (전통 문법)	뜻
`control?.setAttribute(…)`	`if (control) control.setAttribute(…)`	`control` 이 있을 때만

최신 문법	실제 코드 (전통 문법)	뜻
`msg.dataset.hint ??= msg.textContent`	`if (msg.dataset.hint === undefined) msg.dataset.hint = msg.textContent`	비어 있을 때만 넣기
`message ?? msg.dataset.hint`	`message === undefined ? msg.dataset.hint : message`	값이 없으면 대신 쓸 값 (`??` 는 `null` 도 "없음" 으로 봄)
`classList.toggle('is-ok', 조건)`	`remove` 후 조건에 맞으면 `add`	조건이 참이면 붙이고 거짓이면 뗌

문법 풀이

import 문법	뜻
`export function`	다른 파일(`join.js` 등)이 `import { setFieldState } from '../core/auth-ui.js'` 로 가져다 씀

식	뜻
`if (!field) return;`	`field` 가 없으면(`null` 등) 아무것도 하지 않고 끝냄
`field.querySelector('.field__control')`	`field` **안에서** 이 클래스를 가진 첫 요소. 없으면 `null`
`classList.remove('is-ok', 'is-error')`	두 클래스를 한 번에 뗌. 없어도 오류 없음
`setAttribute('aria-invalid', 'true')` · `removeAttribute(…)`	HTML 속성 붙이기 · 떼기
`msg.dataset.hint`	HTML 의 `data-hint="…"` 속성. 이 함수가 처음 부를 때 만들어 안내 글을 보관
`조건 ? A : B`	조건이 참이면 A, 아니면 B (오래된 전통 문법 — 삼항 연산자)
`message === undefined`	세 번째 인자를 **안 넘겼는지**. `''`(빈 글자)를 넘기면 빈 글자로 바꿈

4. 흐름

1. `field` 가 없으면 끝.
2. 상태 표시를 모두 지운다 (`is-ok` · `is-error` · `aria-invalid`).
3. `'ok'` 면 초록, `'error'` 면 빨강 + `aria-invalid`.
4. 문구를 바꾼다 — 넘긴 문구, 없으면 처음 안내 글.

5. 왜 이렇게

선택	이유
먼저 모두 지운 뒤 붙임	`ok` → `error` 로 바뀔 때 `is-ok` 가 남아 초록 · 빨강이 섞이지 않는다
색은 CSS 에, JS 는 클래스만	디자인을 바꿀 때 JS 를 건드리지 않는다. 같은 상태 규칙을 모든 폼(`forms.css`)이 쓴다
`aria-invalid="true"`	색을 볼 수 없는 사람도 화면 읽기 프로그램으로 "잘못된 입력" 을 안다. 문구는 `aria-live="polite"` 칸이라 바뀌면 읽어 준다
처음 안내 글을 `data-hint` 에 보관	"2–20 characters" 같은 안내가 있는 칸에서, 오류를 지우면 안내 글이 돌아온다. HTML 을 고치지 않아도 된다
`field` · `control` · `msg` 가 없어도 오류 없음	칸 모양이 조금 다른 폼(로그인 모달 등)에서도 안전하게 쓸 수 있다

6. 시험 (실제 join.html 을 헤드리스 Chrome 으로 열고 그 안에서 실행 · 11개)

P1-02 의 Node 시험과 달리, 이 함수는 실제 HTML · CSS 가 있어야 하므로 **브라우저 안에서** 시험했다 (Sentry 를 끈 8001 번 개발 서버).

시험	확인한 것
처음 상태	클래스 없음, `aria-invalid` 없음, 문구 빈 글자
ok · error · null · error→ok (4개)	클래스 · `aria-invalid` · 문구가 표대로, 바뀔 때 이전 상태가 남지 않음
실제 밑줄 색	error `rgb(255, 138, 138)` · ok `rgb(127, 224, 168)` · 기본 `rgba(163, 131, 255, 0.45)` — `forms.css` 가 실제로 적용됨
안내 글이 있는 칸 (4개)	error 뒤 null 이면 "2–20 characters" 로 복원, 문구 없이 ok 도 안내 글, 빈 글자를 넘기면 빈 글자
`field` 가 `null`	오류 없음

시험 중 한 번 실패한 것: 처음에는 밑줄 색이 세 상태 모두 같게 나왔다. `forms.css` 의 색 전환 효과(`transition`) 때문에 바꾼 직후에는 이전 색이었다. 함수 문제가 아니라 시험이 너무 빨리 읽은 것이라, 0.6초 기다린 뒤 읽도록 고쳤다.

11/11 통과.